WEEK 3

Object-Oriented Kotlin & Git

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

So far your programs have used separate variables and functions. Real apps model **things**: a User, a Product, a Message, an Order. **Object-oriented programming (OOP)** lets you bundle the data and the actions of each thing into a **class**. Every Android screen, ViewModel and data model you'll write is a class, so this week is very important.

Day	Topic	You will be able to...
Day 1	Classes & objects	Create your own types with properties and functions
Day 2	Constructors & encapsulation	Validate data and hide details with private
Day 3	Inheritance & abstract classes	Share code between related classes, override behaviour
Day 4	Interfaces, data classes & enums	Write the kinds of classes used most in Android
Day 5	Sealed classes, objects & extensions	Model screen states and add functions to existing types
Day 6	Git & GitHub + weekly project	Save code with Git, and build a Library Management System

> **Tip:** Use the **Kotlin Playground** for all of this week's programs. Every example has been run, and the output shown is the real output.

WEEK 3 • DAY 1

Classes & Objects

Today's goal: Understand what classes and objects are, and create your own classes with properties and functions.

1. Class and object: the blueprint and the house

A **class** is a **blueprint** (a design). An **object** is a real thing built from that blueprint. One house plan (class) can be used to build many houses (objects). Each house has its own colour and owner.

Class (blueprint)	Objects (real things)
Student	Anjana (roll 1), Rahul (roll 2), Meera (roll 3)
Phone	Anjana's Pixel, Rahul's Samsung
Product	Rice Rs 60, Milk Rs 28

A class has two kinds of members:

- **Properties**: the data it **has** (name, age, price). They are variables inside the class.
- **Functions** (also called **methods**): the things it can **do** (introduce(), addToCart()).

2. Your first class

A class with properties and a function

```
class Student(val name: String, val rollNumber: Int, var marks: Int) {

    fun introduce() {
        println("Hi, I am $name (roll $rollNumber) and I scored $marks marks.")
    }
}

fun main() {
    val anjana = Student("Anjana", 1, 92)     // create an object
    val rahul = Student("Rahul", 2, 78)       // another object

    anjana.introduce()
    rahul.introduce()

    println(anjana.name)          // read a property with a dot
    rahul.marks = 81              // change a var property
    rahul.introduce()
}
```

Output

```
Hi, I am Anjana (roll 1) and I scored 92 marks.
Hi, I am Rahul (roll 2) and I scored 78 marks.
Anjana
Hi, I am Rahul (roll 2) and I scored 81 marks.
```

Code	Meaning
class Student(...)	Define a new type called Student. Class names start with a **capital letter**
(val name: String, ...)	The **primary constructor**: the values you give when creating an object. **val/var** here makes each one a property
Student("Anjana", 1, 92)	Create ("instantiate") an object. No **new** keyword in Kotlin

Code	Meaning
anjana.introduce()	Call a function on that object, using the dot
rahul.marks = 81	Change a property. Only possible because marks is a **var**

3. Each object has its own data

Objects are independent

```
class Counter(var count: Int = 0) {
    fun increment() {
        count++
    }
}

fun main() {
    val a = Counter()
    val b = Counter()

    a.increment()
    a.increment()
    b.increment()

    println("a = ${a.count}")
    println("b = ${b.count}")
}
```

Output

```
a = 2
b = 1
```

var count: Int = 0 gives the property a **default value**, so you can create a Counter without passing anything.

4. Properties inside the class body

Not every property has to come from the constructor. You can also declare properties inside the class body, including **computed properties** that calculate their value each time you read them.

Body properties and computed properties

```
class Rectangle(val width: Double, val height: Double) {
    val area: Double = width * height          // calculated once, when the object is created

    val isSquare: Boolean
        get() = width == height                // calculated every time you read it

    fun describe() {
        println("Rectangle ${width}x$height, area $area, square: $isSquare")
    }
}

fun main() {
    Rectangle(4.0, 2.5).describe()
    Rectangle(3.0, 3.0).describe()
}
```

Output

```
Rectangle 4.0x2.5, area 10.0, square: false
Rectangle 3.0x3.0, area 9.0, square: true
```

5. Functions that return values and change state

A bank account

```
class BankAccount(val owner: String, var balance: Double = 0.0) {

    fun deposit(amount: Double) {
        balance += amount
        println("$owner deposited Rs $amount. Balance: Rs $balance")
    }

    fun withdraw(amount: Double): Boolean {
        if (amount > balance) {
            println("$owner: not enough money! Balance is Rs $balance")
            return false
        }
        balance -= amount
        println("$owner withdrew Rs $amount. Balance: Rs $balance")
        return true
    }
}

fun main() {
    val account = BankAccount("Anjana")
    account.deposit(5000.0)
    account.withdraw(1200.0)
    val ok = account.withdraw(10000.0)
    println("Second withdrawal worked? $ok")
}
```

Output

```
Anjana deposited Rs 5000.0. Balance: Rs 5000.0
Anjana withdrew Rs 1200.0. Balance: Rs 3800.0
Anjana: not enough money! Balance is Rs 3800.0
Second withdrawal worked? false
```

6. A list of objects

Objects in collections (Week 2 + Week 3)

```
class Product(val name: String, val price: Int, val inStock: Boolean)

fun main() {
    val products = listOf(
        Product("Rice", 60, true),
        Product("Ghee", 650, false),
        Product("Milk", 28, true)
    )

    val available = products.filter { it.inStock }
    println("Available: ${available.map { it.name }}")
    println("Total of available: Rs ${available.sumOf { it.price }}")
    println("Costliest: ${products.maxBy { it.price }.name}")
}
```

Output

```
Available: [Rice, Milk]
Total of available: Rs 88
Costliest: Ghee
```

> **Tip:** This is exactly how Android apps work: a list of objects (messages, products) comes from the server, and you filter, sort and show it on screen.

7. Printing an object

What println shows for a normal class

```
class Book(val title: String)

fun main() {
    val book = Book("Kotlin Basics")
    println(book.title)
    println(book is Book)
}
```

Output

```
Kotlin Basics
true
```

If you print a normal object directly with **println(book)**, you get something unhelpful like *Book@1b6d3586*. On Day 4 you'll learn **data classes**, which print nicely. **is** checks an object's type.

Practice Exercises

1. Create a class **Car(brand, model, year)** with a function **details()** that prints all three. Create 2 cars and call details() on both.

2. Create a class **Circle(radius: Double)** with a computed property **area** (3.14 * radius * radius). Print the area for radius 5.0.

3. Create a class **Lamp** with **var isOn = false** and functions turnOn(), turnOff() and status(). Turn it on and off and print its status each time.

4. Create a class **Movie(title, rating: Double)**. Make a list of 4 movies and print the titles of the movies rated 8.0 or more.

5. Add a **transfer(to: BankAccount, amount: Double)** function to BankAccount that withdraws from one account and deposits into the other.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between a class and an object?

Q2. What are properties and functions in a class?

Q3. How do you create an object in Kotlin?

Q4. Why can you change rahul.marks but not rahul.name?

Q5. What is a computed property?

WEEK 3 • DAY 2

Constructors & Encapsulation

Today's goal: Validate data when objects are created, protect data with private, and control how properties change.

1. The init block

The **init** block runs **automatically** when an object is created. Use it to check or prepare data.

init runs at creation

```
class User(val name: String, val age: Int) {
    init {
        println("Creating user $name")
        require(age >= 0) { "Age cannot be negative" }
    }
}

fun main() {
    val u = User("Anjana", 20)
    println("Created: ${u.name}, ${u.age}")
}
```

Output

```
Creating user Anjana
Created: Anjana, 20
```

require(condition) { message } stops the program with an error if the condition is false. It protects your class from bad data: `User("X", -5)` would crash with "Age cannot be negative".

2. Default and named arguments in constructors

Flexible object creation

```
class Pizza(
    val size: String = "Medium",
    val cheese: Boolean = true,
    val toppings: List<String> = emptyList()
) {
    fun describe() {
        println("$size pizza, cheese: $cheese, toppings: $toppings")
    }
}

fun main() {
    Pizza().describe()
    Pizza("Large").describe()
    Pizza(toppings = listOf("Mushroom", "Corn")).describe()
    Pizza(cheese = false, size = "Small").describe()
}
```

Output

```
Medium pizza, cheese: true, toppings: []
Large pizza, cheese: true, toppings: []
Medium pizza, cheese: true, toppings: [Mushroom, Corn]
Small pizza, cheese: false, toppings: []
```

Tip: This is exactly how Compose functions work: `Text(text = "Hi", fontSize = 20.sp)` uses named arguments with defaults for everything else.

3. Secondary constructors

Another way to create an object

```
class Temperature(val celsius: Double) {
    constructor(fahrenheit: Int) : this((fahrenheit - 32) / 1.8)

    fun show() = println("Temperature: $celsius C")
}

fun main() {
    Temperature(25.0).show()
    Temperature(212).show()
}
```

Output

```
Temperature: 25.0 C
Temperature: 100.0 C
```

A **secondary constructor** must call the primary one with **this(...)**. In practice, default arguments are usually simpler, so secondary constructors are rare in modern Kotlin.

4. Encapsulation: hiding details with private

Encapsulation means an object controls its own data. Other code can't change it directly. It can only use the functions the class provides. This prevents mistakes like setting a balance to a negative number.

Modifier	Who can see it
public (default)	Everyone
private	Only code inside the same class
protected	The class and its subclasses (Day 3)
internal	Code in the same module (the same app)

A safe wallet

```
class Wallet {
    private var balance = 0

    fun add(amount: Int) {
        if (amount > 0) balance += amount
    }

    fun spend(amount: Int): Boolean {
        if (amount > balance) return false
        balance -= amount
        return true
    }

    fun getBalance(): Int = balance
}

fun main() {
    val wallet = Wallet()
    wallet.add(500)
    wallet.add(-100)              // ignored: negative
    println(wallet.spend(200))
    println(wallet.spend(1000))
    println("Balance: ${wallet.getBalance()}")
    // wallet.balance = 99999     // Error: Cannot access 'balance': it is private
}
```

Output

```
true
false
Balance: 300
```

5. private set: read from outside, change only inside

A very common Kotlin pattern: everyone can **read** the property, but only the class can **change** it.

private set

```
class Score {
    var points: Int = 0
        private set

    fun addGoal() {
        points += 10
    }
}

fun main() {
    val score = Score()
    score.addGoal()
    score.addGoal()
    println("Points: ${score.points}")    // reading is allowed
    // score.points = 1000               // Error: the setter is private
}
```

Output

```
Points: 20
```

Tip: You'll see this idea in every ViewModel: `private val _uiState` (only the ViewModel changes it) and a public read-only `uiState` for the screen. It's encapsulation!

6. Custom setters with validation

A setter that checks the value

```
class Student(val name: String) {
    var marks: Int = 0
        set(value) {
            field = value.coerceIn(0, 100)   // keep between 0 and 100
        }
}

fun main() {
    val s = Student("Anjana")
    s.marks = 85
    println(s.marks)
    s.marks = 150
    println(s.marks)
    s.marks = -20
    println(s.marks)
}
```

Output

```
85
100
0
```

Inside a setter, **field** means the actual stored value, and **value** is the new value someone is trying to set. **coerceIn(0, 100)** forces a number into a range.

7. lateinit: "I'll set this later"

lateinit var

```
class Profile {
    lateinit var username: String

    fun load() {
        username = "anjana_dev"      // set later, not in the constructor
    }
}

fun main() {
    val p = Profile()
    p.load()
    println(p.username)
}
```

Output

```
anjana_dev
```

Careful: Reading a lateinit property before setting it crashes with *UninitializedPropertyAccessException*. You'll see lateinit in Android code (for example ViewBinding in Activities), but prefer constructor values or nullable types when you can.

Practice Exercises

1. Create a class **Product(name, price)** with an init block that uses require to make sure the price is greater than 0.

2. Create a class **Thermostat** with a **var temperature** whose setter keeps it between 16 and 30. Test it with 10, 22 and 35.

3. Create a class **VoteCounter** with **votes** that has a private set, and a vote() function. Vote 3 times and print the count.

4. Create a class **Email(address: String)** whose init block requires that the address contains "@". Create one valid email and print it.

5. Create a class **ShoppingCart** with a **private** mutable list of item names, plus addItem(name), removeItem(name) and itemCount().

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. When does the init block run?

Q2. What does private mean?

Q3. What does private set do?

Q4. In a custom setter, what are field and value?

Q5. What happens if you read a lateinit property before setting it?

WEEK 3 • DAY 3

Inheritance & Abstract Classes

Today's goal: Reuse code by building classes on top of other classes, and override behaviour.

1. What is inheritance?

Inheritance lets a class (the **child**, or **subclass**) get all the properties and functions of another class (the **parent**, or **superclass**), and then add or change things. A Dog **is an** Animal, so Dog can inherit from Animal.

Sketch: A class hierarchy

```
            Animal
          (name, eat())
         /      |      \
      Dog      Cat      Bird
   (bark())  (meow())  (fly())
```

2. open and inheritance

In Kotlin, classes are **closed** by default: you can't inherit from them. Add **open** to allow it.

A parent and two children

```
open class Animal(val name: String) {
    fun eat() {
        println("$name is eating")
    }
}

class Dog(name: String) : Animal(name) {
    fun bark() {
        println("$name says Woof!")
    }
}

class Cat(name: String) : Animal(name) {
    fun meow() {
        println("$name says Meow!")
    }
}

fun main() {
    val dog = Dog("Bruno")
    val cat = Cat("Kitty")

    dog.eat()       // inherited from Animal
    dog.bark()      // Dog's own function
    cat.eat()
    cat.meow()
}
```

Output

```
Bruno is eating
Bruno says Woof!
Kitty is eating
Kitty says Meow!
```

Code	Meaning
open class Animal	Other classes are allowed to inherit from Animal
class Dog(name: String) : Animal(name)	Dog inherits from Animal, and passes name to Animal's constructor
dog.eat()	Dog didn't write eat(), but it has it because Dog **is an** Animal

3. Overriding functions

A child can **replace** a parent's function with its own version. The parent's function must be **open**, and the child's must say **override**.

override

```
open class Animal(val name: String) {
    open fun makeSound() {
        println("$name makes a sound")
    }
}

class Dog(name: String) : Animal(name) {
    override fun makeSound() {
        println("$name barks: Woof!")
    }
}

class Cow(name: String) : Animal(name) {
    override fun makeSound() {
        super.makeSound()                // run the parent's version first
        println("$name moos: Mooo!")
    }
}

fun main() {
    Dog("Bruno").makeSound()
    Cow("Lakshmi").makeSound()
}
```

Output

```
Bruno barks: Woof!
Lakshmi makes a sound
Lakshmi moos: Mooo!
```

super.makeSound() calls the parent's version. Use it when you want to **add** to the parent's behaviour instead of replacing it completely.

Tip: You've already seen override in Android: `override fun onCreate(...)` in MainActivity, followed by `super.onCreate(...)`. Now you know what both mean!

4. Polymorphism: one type, many forms

A variable of the parent type can hold **any** child object. When you call an overridden function, the **child's** version runs. This is called **polymorphism**.

A list of different animals

```
open class Animal(val name: String) {
    open fun makeSound() = println("$name makes a sound")
}

class Dog(name: String) : Animal(name) {
    override fun makeSound() = println("$name: Woof!")
}

class Cat(name: String) : Animal(name) {
    override fun makeSound() = println("$name: Meow!")
}

fun main() {
    val animals: List<Animal> = listOf(Dog("Bruno"), Cat("Kitty"), Animal("Unknown"))
    for (animal in animals) {
        animal.makeSound()       // each one uses its own version
    }
}
```

Output

```
Bruno: Woof!
Kitty: Meow!
Unknown makes a sound
```

5. Abstract classes

Sometimes the parent is only an idea: a "Shape" with no real shape. An **abstract class** can't be created directly. It defines **abstract** members that every child **must** implement.

Abstract Shape

```
abstract class Shape(val name: String) {
    abstract fun area(): Double                // no body: children must write it

    fun describe() {                           // normal function, shared by all
        println("$name has area ${"%.2f".format(area())}")
    }
}

class Circle(val radius: Double) : Shape("Circle") {
    override fun area() = 3.14159 * radius * radius
}

class Rectangle(val width: Double, val height: Double) : Shape("Rectangle") {
    override fun area() = width * height
}

fun main() {
    val shapes = listOf(Circle(2.0), Rectangle(3.0, 4.5))
    shapes.forEach { it.describe() }
    // val s = Shape("x")   // Error: Cannot create an instance of an abstract class
}
```

Output

```
Circle has area 12.57
Rectangle has area 13.50
```

"%.2f".format(x) shows a Double with 2 decimal places.

6. protected members

protected: visible to children only

```
open class Employee(val name: String) {
    protected open val baseSalary = 30000

    open fun salary(): Int = baseSalary
}

class Manager(name: String) : Employee(name) {
    override fun salary(): Int = baseSalary + 20000    // the child can use baseSalary
}

fun main() {
    println("${Employee("Ravi").name}: ${Employee("Ravi").salary()}")
    println("${Manager("Meera").name}: ${Manager("Meera").salary()}")
    // Employee("Ravi").baseSalary   // Error: it is protected
}
```

Output

```
Ravi: 30000
Meera: 50000
```

7. When to use inheritance

Use inheritance when...	Avoid it when...
There is a true **"is a"** relationship (a Dog is an Animal)	You only want to reuse a little code
A framework requires it (Activity, ViewModel)	The hierarchy would get deep (more than 2 or 3 levels)
Children share real behaviour	Classes only share a "can do" ability: use an interface instead (Day 4)

Practice Exercises

1. Create an open class **Vehicle(name, wheels)** with an open function info(). Create Bike and Car subclasses that override info().

2. Create an abstract class **Employee(name)** with an abstract function salary(). Create FullTime (fixed 40000) and PartTime (hours * 500) subclasses. Print both salaries.

3. Put a Circle, a Rectangle and a new **Triangle(base, height)** (area = 0.5 * base * height) in a list and print each area.

4. Create an open class **Notification(title)** with open fun send(). Create EmailNotification and SmsNotification that call super.send() and then print their own message.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why do you need open before a class in Kotlin?

Q2. What does override mean?

Q3. What does super.function() do?

Q4. Can you create an object of an abstract class?

Q5. What is polymorphism?

WEEK 3 • DAY 4

Interfaces, Data Classes & Enums

Today's goal: Use interfaces for shared abilities, data classes for data models, and enum classes for fixed choices.

1. Interfaces: "can do" abilities

An **interface** describes something a class **can do**, without saying how. Different, unrelated classes can share the same ability. A class can implement **many** interfaces, but inherit from only **one** class.

Interfaces

```
interface Clickable {
    fun onClick()                               // must be implemented
    fun showRipple() = println("Ripple effect") // has a default implementation
}

interface Shareable {
    fun share(): String
}

class Photo(val fileName: String) : Clickable, Shareable {
    override fun onClick() = println("Opening photo $fileName")
    override fun share() = "Sharing $fileName"
}

class Button(val label: String) : Clickable {
    override fun onClick() = println("Button '$label' clicked")
}

fun main() {
    val items: List<Clickable> = listOf(Photo("beach.jpg"), Button("Save"))
    for (item in items) {
        item.onClick()
        item.showRipple()
    }
    println(Photo("cat.png").share())
}
```

Output

```
Opening photo beach.jpg
Ripple effect
Button 'Save' clicked
Ripple effect
Sharing cat.png
```

	Abstract class	Interface
How many can a class use?	Only one	Many
Can store state (properties with values)?	Yes	No (only abstract properties)
Constructor?	Yes	No
Meaning	"is a" (Dog is an Animal)	"can do" (Photo can be shared)

Tip: In Android, repositories are often interfaces: `interface TaskRepository { suspend fun getTasks(): List<Task> }`. The real app uses a network version, and tests use a fake version.

2. Data classes: classes that hold data

Most classes in an app just hold data: a User, a Message, a Product. Add **data** before class and Kotlin automatically writes useful functions for you.

data class superpowers

```
data class User(val id: Int, val name: String, val email: String)

fun main() {
    val user1 = User(1, "Anjana", "anjana@example.com")
    val user2 = User(1, "Anjana", "anjana@example.com")

    println(user1)                    // 1. readable toString()
    println(user1 == user2)           // 2. compares the values, not the memory address

    val updated = user1.copy(email = "anjana@new.com")   // 3. copy with changes
    println(updated)
    println(user1)                    // the original is unchanged

    val (id, name, email) = user1     // 4. destructuring
    println("$id | $name | $email")
}
```

Output

```
User(id=1, name=Anjana, email=anjana@example.com)
true
User(id=1, name=Anjana, email=anjana@new.com)
User(id=1, name=Anjana, email=anjana@example.com)
1 | Anjana | anjana@example.com
```

Feature	What you get
toString()	Prints User(id=1, name=Anjana, ...) instead of User@1b6d3586
equals() / ==	Two objects with the same values are equal
copy()	A new object with some values changed. The original stays the same (immutability)
Destructuring	val (id, name) = user
hashCode()	Works correctly in sets and as map keys

Normal class vs data class

```
class PlainPoint(val x: Int, val y: Int)
data class DataPoint(val x: Int, val y: Int)

fun main() {
    println(PlainPoint(1, 2) == PlainPoint(1, 2))
    println(DataPoint(1, 2) == DataPoint(1, 2))
    println(DataPoint(1, 2))
}
```

Output

```
false
true
DataPoint(x=1, y=2)
```

Tip: In Android, almost every model (UiState, API responses, database rows) is a **data class with val properties**. To change one, you create a new copy: `state.copy(isLoading = true)`. You saw this in the Compose guide!

3. Enum classes: a fixed set of choices

When a value can only be one of a few fixed options (days, sizes, order status), use an **enum class**.

enum with when

```
enum class OrderStatus {
    PLACED, PACKED, SHIPPED, DELIVERED, CANCELLED
}

fun statusMessage(status: OrderStatus): String = when (status) {
    OrderStatus.PLACED -> "We received your order"
    OrderStatus.PACKED -> "Your order is packed"
    OrderStatus.SHIPPED -> "On the way!"
    OrderStatus.DELIVERED -> "Delivered. Enjoy!"
    OrderStatus.CANCELLED -> "Order cancelled"
}

fun main() {
    val status = OrderStatus.SHIPPED
    println(statusMessage(status))
    println(status.name)              // the name as text
    println(status.ordinal)           // its position, starting at 0
    println(OrderStatus.entries.size) // how many options exist
}
```

Output

```
On the way!
SHIPPED
2
5
```

Notice there's **no else** in the when: Kotlin knows every possible value, so it checks you handled all of them. If you add a new status later, the compiler shows you every when you need to update.

Enums with properties

```
enum class Size(val label: String, val extraPrice: Int) {
    SMALL("Small", 0),
    MEDIUM("Medium", 40),
    LARGE("Large", 80)
}

fun main() {
    for (size in Size.entries) {
        println("${size.label}: +Rs ${size.extraPrice}")
    }
}
```

Output

```
Small: +Rs 0
Medium: +Rs 40
Large: +Rs 80
```

Practice Exercises

1. Create an interface **Payable** with fun pay(amount: Double). Implement it in UpiPayment and CardPayment, each printing a different message.

2. Create a data class **Book(title, author, price)**. Make a book, copy it with a new price, and print both. Check whether two equal books are ==.

3. Create an enum class **TrafficLight** (RED, YELLOW, GREEN) and a function that returns the action for each using when.

4. Create an enum class **Planet(gravity: Double)** for EARTH (1.0), MARS (0.38) and JUPITER (2.53). Print your weight (60 kg) on each.

5. Create a list of data class **Employee(name, department, salary)** objects. Group them by department and print the total salary of each department.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. How many interfaces can a class implement?

Q2. Name 3 things a data class gives you automatically.

Q3. What does copy() do?

Q4. When should you use an enum class?

Q5. Why doesn't a when over all enum values need else?

WEEK 3 • DAY 5

Sealed Classes, Objects & Extensions

Today's goal: Model states with sealed classes, use object and companion object, and add functions to existing types.

1. Sealed classes: a fixed set of types

An enum is a fixed set of **values**. A **sealed class** (or sealed interface) is a fixed set of **types**, and each type can carry **different data**. It's perfect for screen states and results.

Loading, success and error

```
sealed interface ScreenState {
    data object Loading : ScreenState
    data class Success(val items: List<String>) : ScreenState
    data class Error(val message: String) : ScreenState
}

fun render(state: ScreenState) {
    when (state) {
        is ScreenState.Loading -> println("Showing a spinner...")
        is ScreenState.Success -> println("Showing ${state.items.size} items: ${state.items}")
        is ScreenState.Error -> println("Showing error: ${state.message}")
    }
}

fun main() {
    render(ScreenState.Loading)
    render(ScreenState.Success(listOf("Milk", "Bread")))
    render(ScreenState.Error("No internet connection"))
}
```

Output

```
Showing a spinner...
Showing 2 items: [Milk, Bread]
Showing error: No internet connection
```

Part	Meaning
sealed interface ScreenState	Only the types declared here can be a ScreenState
data object Loading	A single object: Loading has no data
data class Success(val items)	Success carries a list
data class Error(val message)	Error carries a message
is ScreenState.Success -> state.items	After the is check, Kotlin smart-casts, so you can use items

Tip: This is the exact pattern Android teams use for **UiState** (Chapter 8 of your Compose guide). Learn it well!

A Result type for operations

```
sealed class LoginResult {
    data class Success(val userName: String) : LoginResult()
    data class Failure(val reason: String) : LoginResult()
}

fun login(email: String, password: String): LoginResult {
    return if (email == "anjana@example.com" && password == "kotlin123") {
        LoginResult.Success("Anjana")
    } else {
        LoginResult.Failure("Wrong email or password")
    }
}

fun main() {
    for (attempt in listOf("kotlin123", "wrong")) {
        when (val result = login("anjana@example.com", attempt)) {
            is LoginResult.Success -> println("Welcome, ${result.userName}!")
            is LoginResult.Failure -> println("Login failed: ${result.reason}")
        }
    }
}
```

Output

```
Welcome, Anjana!
Login failed: Wrong email or password
```

2. object: exactly one instance

Use **object** when you need exactly **one** instance in the whole app, for example app settings or a logger. This is called a **singleton**.

A singleton

```
object AppSettings {
    var darkMode = false
    const val APP_VERSION = "1.0.0"

    fun toggleDarkMode() {
        darkMode = !darkMode
        println("Dark mode is now $darkMode")
    }
}

fun main() {
    println("Version ${AppSettings.APP_VERSION}")
    AppSettings.toggleDarkMode()
    AppSettings.toggleDarkMode()
}
```

Output

```
Version 1.0.0
Dark mode is now true
Dark mode is now false
```

You don't create an object with brackets: you use it directly by name. **const val** is a compile-time constant (a fixed String or number).

3. companion object: things that belong to the class

Factory functions and constants

```
class User private constructor(val name: String, val isGuest: Boolean) {

    companion object {
        const val MAX_NAME_LENGTH = 20

        fun create(name: String): User = User(name.take(MAX_NAME_LENGTH), isGuest = false)
        fun guest(): User = User("Guest", isGuest = true)
    }
}

fun main() {
    val u = User.create("Anjana")
    val g = User.guest()
    println("${u.name}, guest: ${u.isGuest}")
    println("${g.name}, guest: ${g.isGuest}")
    println("Max length: ${User.MAX_NAME_LENGTH}")
}
```

Output

```
Anjana, guest: false
Guest, guest: true
Max length: 20
```

Members of a **companion object** are called on the **class name** (User.guest()), not on an object. The **private constructor** forces everyone to use the factory functions.

4. Extension functions: add functions to existing types

You can add your own functions to **any** type, even String or Int, without changing its code. Inside, **this** is the value the function is called on.

Extension functions and properties

```
fun String.isValidEmail(): Boolean = this.contains("@") && this.contains(".")

fun Int.toRupees(): String = "Rs $this"

fun String.initials(): String =
    this.split(" ").filter { it.isNotEmpty() }.joinToString("") { it.first().uppercase() }

val Int.isEven: Boolean
    get() = this % 2 == 0

fun main() {
    println("anjana@example.com".isValidEmail())
    println("anjana".isValidEmail())
    println(450.toRupees())
    println("anjana menon".initials())
    println(7.isEven)
}
```

Output

```
true
false
Rs 450
AM
false
```

Tip: Many Kotlin functions you already used are extensions: **filter**, **map** and **sumOf** on lists. Compose uses them too, like **Modifier.padding()**.

5. Scope functions: let, apply, also

Common scope functions

```
data class Order(var item: String = "", var quantity: Int = 0, var note: String? = null)

fun main() {
    // apply: configure an object, returns the object
    val order = Order().apply {
        item = "Biryani"
        quantity = 2
    }
    println(order)

    // let: run code on a non-null value
    order.note?.let { println("Note: $it") }      // note is null, so this doesn't run
    order.note = "Less spicy"
    order.note?.let { println("Note: $it") }

    // also: do something extra, returns the object
    val saved = order.also { println("Saving ${it.item}...") }
    println(saved.quantity)
}
```

Output

```
Order(item=Biryani, quantity=2, note=null)
Note: Less spicy
Saving Biryani...
2
```

Function	Inside, the object is...	Returns	Typical use
let	it	the lambda result	Null checks: x?.let { }
apply	this	the object	Setting up an object
also	it	the object	Logging or side effects
run / with	this	the lambda result	Computing a result from an object

Practice Exercises

1. Create a sealed interface **PaymentResult** with Success(amount), Failed(reason) and Pending. Write a function that prints a message for each.
2. Create an **object Logger** with a function log(message) that prints "[LOG] message" and counts how many messages were logged.
3. Write an extension function **String.capitalizeWords()** that turns "hello kotlin world" into "Hello Kotlin World".
4. Write an extension property **Double.withGst** that returns the value plus 18%. Print 100.0.withGst.
5. Give a class **Temperature(celsius)** a companion object with fromFahrenheit(f: Double). Create one from 98.6 F and print the Celsius value.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is the difference between an enum and a sealed class?

Q2. Why are sealed classes great for screen states?

Q3. What is an object declaration used for?

Q4. How do you call a function in a companion object?

Q5. Inside an extension function, what does this refer to?

WEEK 3 • DAY 6

Git & GitHub + Weekly Project

Today's goal: Save and share your code with Git, and build a Library Management System using everything from this week.

1. What is Git?

Git is a tool that saves **snapshots** (versions) of your code. You can go back to any old version, see what changed, and work with a team without overwriting each other's code. **GitHub** is a website that stores Git projects online. Every Android team uses Git every day.

Word	Meaning
Repository (repo)	A project folder tracked by Git
Commit	A saved snapshot of your changes, with a message
Branch	A separate line of work (for example, one feature), so main stays safe
Push / Pull	Upload your commits to GitHub / download others' commits
Pull request (PR)	A request to merge your branch into main, so teammates can review it
Merge conflict	Two people changed the same lines. Git asks you to choose

2. One-time setup

- Install Git from **git-scm.com** (Android Studio can also use it).
- Tell Git who you are (in a terminal): `git config --global user.name "Anjana"` and `git config --global user.email "you@example.com"`
- Create a free **GitHub** account if you don't have one.

3. The daily Git workflow

Sketch: How your changes travel

```
your files  --git add-->  staging area  --git commit-->  local repo  --git push-->  GitHub
(working)                 (ready to save)                (history)                  (online)
                                                                      <--git pull--
```

Command	What it does
git init	Start tracking the current folder with Git (once per project)
git status	Show changed files. Run it often!
git add .	Stage all changes (or git add File.kt for one file)
git commit -m "Add Book class"	Save a snapshot with a message
git log --oneline	Show the history of commits
git checkout -b feature/library	Create a new branch and switch to it
git switch main	Switch back to the main branch
git remote add origin <url>	Connect your local repo to a GitHub repo (once)
git push -u origin feature/library	Upload your branch to GitHub
git pull	Download and merge the latest changes

4. Using Git inside Android Studio

- **VCS / Git** → **Enable Version Control Integration** (once per project), or **Share Project on GitHub**.
- Changed files turn **blue**, and new files turn **green** (or red if they're not added yet).
- **Ctrl + K** opens the Commit window: tick files, write a message, click Commit.
- **Ctrl + Shift + K** pushes. **Ctrl + T** pulls (updates the project).
- The branch name is shown at the top (or bottom right). Click it to create or switch branches.

5. Writing good commit messages

Good	Bad
Add Book and Member data classes	update
Fix crash when borrowing a missing book	fixed stuff
Show available books sorted by title	changes

Tip: Commit small and often, with a message that completes the sentence *"This commit will..."*. Never commit passwords, API keys or the **build/** folder (the **.gitignore** file tells Git what to skip).

PART 2: Project: Library Management System

Build a library system that uses **every topic from this week**:

Feature	Week 3 topic used
Genre: FICTION, SCIENCE...	Enum class (Day 4)
Book and Member	Data classes (Day 4)
Success / Failure results	Sealed interface (Day 5)
Library hides its lists and controls borrowing	Class with private properties: encapsulation (Days 1-2)
Find, filter, group books	Collections and lambdas (Week 2)
?: return when something isn't found	Null safety (Week 2)

Library Management System

```
enum class Genre { FICTION, SCIENCE, HISTORY, TECHNOLOGY }

data class Book(val id: Int, val title: String, val author: String, val genre: Genre)

data class Member(val id: Int, val name: String)

sealed interface LibraryResult {
    data class Success(val message: String) : LibraryResult
    data class Failure(val reason: String) : LibraryResult
}

class Library {
    private val books = mutableListOf<Book>()
    private val members = mutableListOf<Member>()
    private val borrowed = mutableMapOf<Int, Int>()   // bookId -> memberId

    fun addBook(book: Book) = books.add(book)
    fun addMember(member: Member) = members.add(member)

    fun borrow(bookId: Int, memberId: Int): LibraryResult {
        val book = books.find { it.id == bookId }
            ?: return LibraryResult.Failure("No book with id $bookId")
        val member = members.find { it.id == memberId }
            ?: return LibraryResult.Failure("No member with id $memberId")
```

```
        if (bookId in borrowed) return LibraryResult.Failure("'${book.title}' is already borrowed")
        if (borrowed.values.count { it == memberId } >= 2) {
            return LibraryResult.Failure("${member.name} already has 2 books")
        }
        borrowed[bookId] = memberId
        return LibraryResult.Success("${member.name} borrowed '${book.title}'")
    }

    fun returnBook(bookId: Int): LibraryResult {
        val book = books.find { it.id == bookId }
            ?: return LibraryResult.Failure("No book with id $bookId")
        return if (borrowed.remove(bookId) != null) {
            LibraryResult.Success("'${book.title}' returned")
        } else {
            LibraryResult.Failure("'${book.title}' was not borrowed")
        }
    }

    fun availableBooks(): List<Book> = books.filter { it.id !in borrowed }

    fun booksByGenre(): Map<Genre, List<String>> =
        books.groupBy { it.genre }.mapValues { entry -> entry.value.map { it.title } }

    fun search(text: String): List<Book> =
        books.filter { it.title.contains(text, ignoreCase = true) || it.author.contains(text, ignoreCase = true) }
}

fun printResult(result: LibraryResult) {
    when (result) {
        is LibraryResult.Success -> println("OK: ${result.message}")
        is LibraryResult.Failure -> println("FAILED: ${result.reason}")
    }
}

fun main() {
    val library = Library()
    library.addBook(Book(1, "Aadujeevitham", "Benyamin", Genre.FICTION))
    library.addBook(Book(2, "A Brief History of Time", "Stephen Hawking", Genre.SCIENCE))
    library.addBook(Book(3, "Wings of Fire", "A. P. J. Abdul Kalam", Genre.HISTORY))
    library.addBook(Book(4, "Kotlin in Action", "Dmitry Jemerov", Genre.TECHNOLOGY))
    library.addMember(Member(1, "Anjana"))
    library.addMember(Member(2, "Rahul"))

    printResult(library.borrow(4, 1))
    printResult(library.borrow(4, 2))       // already borrowed
    printResult(library.borrow(3, 1))
    printResult(library.borrow(1, 1))       // Anjana already has 2
    printResult(library.borrow(9, 1))       // no such book
    printResult(library.returnBook(4))
    printResult(library.returnBook(4))      // not borrowed any more

    println("Available: ${library.availableBooks().map { it.title }}")
    println("By genre: ${library.booksByGenre()}")
    println("Search 'kalam': ${library.search("kalam").map { it.title }}")
}
```

Output

```
OK: Anjana borrowed 'Kotlin in Action'
FAILED: 'Kotlin in Action' is already borrowed
OK: Anjana borrowed 'Wings of Fire'
FAILED: Anjana already has 2 books
FAILED: No book with id 9
OK: 'Kotlin in Action' returned
FAILED: 'Kotlin in Action' was not borrowed
Available: [Aadujeevitham, A Brief History of Time, Kotlin in Action]
By genre: {FICTION=[Aadujeevitham], SCIENCE=[A Brief History of Time], HISTORY=[Wings of Fire], TECHNOLOGY=[Kotlin in Action]}
Search 'kalam': [Wings of Fire]
```

Understand the tricky lines

Code	What it does
books.find { ... } ?: return LibraryResult.Failure(...)	If find gives null, return a failure immediately (Elvis + return)
borrowed = mutableMapOf<Int, Int>()	Remembers which member has each book: bookId to memberId
if (bookId in borrowed)	in on a map checks whether the key exists
borrowed.values.count { it == memberId }	How many books this member has
borrowed.remove(bookId) != null	remove returns the old value, or null if the key wasn't there
groupBy { it.genre }.mapValues { ... }	Groups books by genre, then keeps only the titles

Project challenges

- Add a **membersWithBooks()** function that returns a map of member names to the titles they borrowed.
- Add a **dueDays** property to borrowing, and a function that lists overdue books.
- Make an interface **Searchable** with fun search(text: String) and have Library implement it.
- Put the project in a new GitHub repository using the Git workflow from Part 1, with at least 3 meaningful commits.

Week 3 checklist

I can...	Done?
Create classes with properties, functions and constructors	[ ]
Use init, default arguments and require for validation	[ ]
Protect data with private and private set	[ ]
Use inheritance with open, override and super	[ ]
Write abstract classes and interfaces, and explain the difference	[ ]
Use data classes (toString, ==, copy) and enum classes	[ ]
Model states with a sealed interface and handle them with when	[ ]
Use object, companion object and extension functions	[ ]
Use Git: add, commit, push, branch	[ ]
Build the Library Management System	[ ]

Quick Quiz

Q1. What is the difference between git commit and git push?

Q2. Why do teams work on branches?

Q3. Which file tells Git which files to ignore?

Q4. Why does the Library keep its lists private?

Q5. Why is LibraryResult a sealed interface?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Classes & Objects

Exercise 1

```
class Car(val brand: String, val model: String, val year: Int) {
    fun details() = println("$year $brand $model")
}

fun main() {
    Car("Maruti", "Swift", 2022).details()
    Car("Tata", "Nexon", 2024).details()
}
```

Output

```
2022 Maruti Swift
2024 Tata Nexon
```

Exercise 2

```
class Circle(val radius: Double) {
    val area: Double
        get() = 3.14 * radius * radius
}

fun main() {
    println(Circle(5.0).area)
}
```

Output

```
78.5
```

Exercise 3

```
class Lamp {
    var isOn = false
    fun turnOn() { isOn = true }
    fun turnOff() { isOn = false }
    fun status() = println(if (isOn) "Lamp is ON" else "Lamp is OFF")
}

fun main() {
    val lamp = Lamp()
    lamp.status()
    lamp.turnOn()
    lamp.status()
    lamp.turnOff()
    lamp.status()
}
```

Output

```
Lamp is OFF
Lamp is ON
Lamp is OFF
```

Exercise 4

```
class Movie(val title: String, val rating: Double)

fun main() {
    val movies = listOf(
        Movie("Drishyam", 8.6), Movie("Premam", 8.3),
        Movie("Average Movie", 6.1), Movie("Kumbalangi Nights", 8.5)
    )
    println(movies.filter { it.rating >= 8.0 }.map { it.title })
}
```

Output

```
[Drishyam, Premam, Kumbalangi Nights]
```

Exercise 5

```
class BankAccount(val owner: String, var balance: Double = 0.0) {
    fun deposit(amount: Double) { balance += amount }
    fun withdraw(amount: Double): Boolean {
        if (amount > balance) return false
        balance -= amount
        return true
    }
    fun transfer(to: BankAccount, amount: Double) {
        if (withdraw(amount)) {
            to.deposit(amount)
            println("Transferred Rs $amount from $owner to ${to.owner}")
        } else {
            println("Transfer failed: not enough money")
        }
    }
}

fun main() {
    val a = BankAccount("Anjana", 1000.0)
    val b = BankAccount("Rahul")
    a.transfer(b, 400.0)
    a.transfer(b, 5000.0)
    println("${a.balance} ${b.balance}")
}
```

Output

```
Transferred Rs 400.0 from Anjana to Rahul
Transfer failed: not enough money
600.0 400.0
```

Quiz answers

Q1. A class is a blueprint; an object is a real instance created from it.

Q2. Properties are the data an object has; functions are the actions it can do.

Q3. Call the class like a function: Student("Anjana", 1, 92). There is no new keyword.

Q4. marks is declared with var; name is declared with val.

Q5. A property with a custom get() that calculates its value every time it is read.

Day 2: Constructors & Encapsulation

Exercise 1

```
class Product(val name: String, val price: Double) {
    init {
        require(price > 0) { "Price must be greater than 0" }
    }
}

fun main() {
    val p = Product("Pen", 10.0)
    println("${p.name}: ${p.price}")
}
```

Output

```
Pen: 10.0
```

Exercise 2

```
class Thermostat {
    var temperature: Int = 22
        set(value) {
            field = value.coerceIn(16, 30)
        }
}

fun main() {
    val t = Thermostat()
    for (v in listOf(10, 22, 35)) {
        t.temperature = v
        println(t.temperature)
    }
}
```

Output

```
16
22
30
```

Exercise 3

```
class VoteCounter {
    var votes = 0
        private set
    fun vote() { votes++ }
}

fun main() {
    val c = VoteCounter()
    repeat(3) { c.vote() }
    println("Votes: ${c.votes}")
}
```

Output

```
Votes: 3
```

Exercise 4

```
class Email(val address: String) {
    init {
        require("@" in address) { "Invalid email: $address" }
    }
}

fun main() {
    println(Email("anjana@example.com").address)
}
```

Output

```
anjana@example.com
```

Exercise 5

```
class ShoppingCart {
    private val items = mutableListOf<String>()
    fun addItem(name: String) = items.add(name)
    fun removeItem(name: String) = items.remove(name)
    fun itemCount() = items.size
}

fun main() {
    val cart = ShoppingCart()
    cart.addItem("Milk")
    cart.addItem("Bread")
    cart.addItem("Eggs")
    cart.removeItem("Bread")
    println("Items: ${cart.itemCount()}")
}
```

Output

```
Items: 2
```

Quiz answers

Q1. Automatically, every time an object is created.

Q2. Only code inside the same class can see or use it.

Q3. Everyone can read the property, but only the class can change it.

Q4. field is the stored value; value is the new value being set.

Q5. The program crashes with UninitializedPropertyAccessException.

Day 3: Inheritance & Abstract Classes

Exercise 1

```
open class Vehicle(val name: String, val wheels: Int) {
    open fun info() = println("$name has $wheels wheels")
}

class Bike(name: String) : Vehicle(name, 2) {
    override fun info() = println("Bike $name: $wheels wheels, no roof")
}

class Car(name: String) : Vehicle(name, 4) {
    override fun info() = println("Car $name: $wheels wheels, seats 5")
}

fun main() {
    Bike("Splendor").info()
    Car("Swift").info()
}
```

Output

```
Bike Splendor: 2 wheels, no roof
Car Swift: 4 wheels, seats 5
```

Exercise 2

```
abstract class Employee(val name: String) {
    abstract fun salary(): Int
}

class FullTime(name: String) : Employee(name) {
    override fun salary() = 40000
}

class PartTime(name: String, val hours: Int) : Employee(name) {
    override fun salary() = hours * 500
}

fun main() {
    val staff = listOf(FullTime("Anjana"), PartTime("Rahul", 30))
    staff.forEach { println("${it.name}: ${it.salary()}") }
}
```

Output

```
Anjana: 40000
Rahul: 15000
```

Exercise 3

```
abstract class Shape {
    abstract fun area(): Double
}
class Circle(val r: Double) : Shape() { override fun area() = 3.14 * r * r }
class Rectangle(val w: Double, val h: Double) : Shape() { override fun area() = w * h }
class Triangle(val base: Double, val height: Double) : Shape() {
    override fun area() = 0.5 * base * height
}

fun main() {
    val shapes = listOf(Circle(1.0), Rectangle(2.0, 3.0), Triangle(4.0, 5.0))
    shapes.forEach { println(it.area()) }
}
```

Output

```
3.14
6.0
10.0
```

Exercise 4

```
open class Notification(val title: String) {
    open fun send() = println("Sending notification: $title")
}

class EmailNotification(title: String) : Notification(title) {
    override fun send() {
        super.send()
        println("...by email")
    }
}

class SmsNotification(title: String) : Notification(title) {
    override fun send() {
        super.send()
        println("...by SMS")
    }
}

fun main() {
    EmailNotification("Order shipped").send()
    SmsNotification("OTP 1234").send()
}
```

Output

```
Sending notification: Order shipped
...by email
Sending notification: OTP 1234
...by SMS
```

Quiz answers

Q1. Classes are closed (final) by default; open allows other classes to inherit from it.

Q2. The child class replaces a parent's open function with its own version.

Q3. It calls the parent's version of the function.

Q4. No. Only its non-abstract children can be created.

Q5. A parent-type variable can hold any child object, and calling an overridden function runs the child's version.

Day 4: Interfaces, Data Classes & Enums

Exercise 1

```
interface Payable {
    fun pay(amount: Double)
}

class UpiPayment(val upiId: String) : Payable {
    override fun pay(amount: Double) = println("Paid Rs $amount via UPI ($upiId)")
}

class CardPayment(val last4: String) : Payable {
    override fun pay(amount: Double) = println("Paid Rs $amount with card ending $last4")
}

fun main() {
    val methods: List<Payable> = listOf(UpiPayment("anjana@upi"), CardPayment("4321"))
    methods.forEach { it.pay(250.0) }
}
```

Output

```
Paid Rs 250.0 via UPI (anjana@upi)
Paid Rs 250.0 with card ending 4321
```

Exercise 2

```
data class Book(val title: String, val author: String, val price: Int)

fun main() {
    val b1 = Book("Wings of Fire", "Kalam", 350)
    val b2 = b1.copy(price = 299)
    println(b1)
    println(b2)
    println(b1 == Book("Wings of Fire", "Kalam", 350))
}
```

Output

```
Book(title=Wings of Fire, author=Kalam, price=350)
Book(title=Wings of Fire, author=Kalam, price=299)
true
```

Exercise 3

```
enum class TrafficLight { RED, YELLOW, GREEN }

fun action(light: TrafficLight) = when (light) {
    TrafficLight.RED -> "Stop"
    TrafficLight.YELLOW -> "Get ready"
    TrafficLight.GREEN -> "Go"
}

fun main() {
    TrafficLight.entries.forEach { println("$it: ${action(it)}") }
}
```

Output

```
RED: Stop
YELLOW: Get ready
GREEN: Go
```

Exercise 4

```
enum class Planet(val gravity: Double) {
    EARTH(1.0), MARS(0.38), JUPITER(2.53)
}

fun main() {
    val weight = 60.0
    for (p in Planet.entries) {
        println("${p.name}: ${"%.1f".format(weight * p.gravity)} kg")
    }
}
```

Output

```
EARTH: 60.0 kg
MARS: 22.8 kg
JUPITER: 151.8 kg
```

Exercise 5

```
data class Employee(val name: String, val department: String, val salary: Int)

fun main() {
    val staff = listOf(
        Employee("Anjana", "Android", 50000),
        Employee("Rahul", "Android", 45000),
        Employee("Meera", "Design", 40000),
        Employee("John", "Testing", 35000)
    )
    staff.groupBy { it.department }.forEach { (dept, people) ->
        println("$dept: ${people.sumOf { it.salary }}")
    }
}
```

Output

```
Android: 95000
Design: 40000
Testing: 35000
```

Quiz answers

Q1. As many as it needs (but it can extend only one class).

Q2. toString(), equals() (==), copy(), hashCode() and destructuring.

Q3. It creates a new object with some properties changed, leaving the original unchanged.

Q4. When a value can only be one of a fixed set of options.

Q5. Kotlin knows all the possible values and checks that every one is handled.

Day 5: Sealed Classes, Objects & Extensions

Exercise 1

```
sealed interface PaymentResult {
    data class Success(val amount: Double) : PaymentResult
    data class Failed(val reason: String) : PaymentResult
    data object Pending : PaymentResult
}

fun show(result: PaymentResult) = when (result) {
    is PaymentResult.Success -> println("Paid Rs ${result.amount}")
    is PaymentResult.Failed -> println("Payment failed: ${result.reason}")
    PaymentResult.Pending -> println("Payment pending...")
}

fun main() {
    show(PaymentResult.Success(499.0))
    show(PaymentResult.Failed("Insufficient balance"))
    show(PaymentResult.Pending)
}
```

Output

```
Paid Rs 499.0
Payment failed: Insufficient balance
Payment pending...
```

Exercise 2

```
object Logger {
    var count = 0
        private set

    fun log(message: String) {
        count++
        println("[LOG] $message")
    }
}

fun main() {
    Logger.log("App started")
    Logger.log("User logged in")
    println("Messages logged: ${Logger.count}")
}
```

Output

```
[LOG] App started
[LOG] User logged in
Messages logged: 2
```

Exercise 3

```
fun String.capitalizeWords(): String =
    split(" ").joinToString(" ") { word -> word.replaceFirstChar { it.uppercase() } }

fun main() {
    println("hello kotlin world".capitalizeWords())
}
```

Output

```
Hello Kotlin World
```

Exercise 4

```
val Double.withGst: Double
    get() = this * 1.18

fun main() {
    println(100.0.withGst)
}
```

Output

```
118.0
```

Exercise 5

```
class Temperature(val celsius: Double) {
    companion object {
        fun fromFahrenheit(f: Double) = Temperature((f - 32) * 5 / 9)
    }
}

fun main() {
    val t = Temperature.fromFahrenheit(98.6)
    println("%.1f C".format(t.celsius))
}
```

Output

```
37.0 C
```

Quiz answers

Q1. An enum is a fixed set of values; a sealed class is a fixed set of types, and each type can hold different data.

Q2. Each state (Loading, Success, Error) holds its own data, and when forces you to handle all of them.

Q3. A singleton: exactly one instance in the whole app.

Q4. Using the class name, for example User.guest().

Q5. The value the function is called on.

Day 6: Git & GitHub + Weekly Project

Quiz answers

Q1. commit saves a snapshot locally; push uploads your commits to GitHub.

Q2. To develop features separately without breaking the main code until they are reviewed.

Q3. .gitignore

Q4. Encapsulation: other code must use borrow/returnBook, so the rules (2-book limit) are always enforced.

Q5. A result is either Success or Failure, each with its own data, and when must handle both.

Next: When you finish this week, tell your mentor and ask for the **Week 4** material. Great work, Anjana!